Priority Queues and Heaps

Prof. Rohit Pal Singh

Department of Computer Science

University of Wisconsin, Milwaukee

September 23, 2024

Readings: Chapter 2.4

Priority Queue ADT

- A priority queue stores a collection of items
- An item is a pair (*key*, *element*)
- Main methods of the Priority Queue ADT
  - **insertItem**(k, o)
    inserts an item with key k and element o
  - **removeMin**()
    removes the item with smallest key and returns its element
- Additional methods
  - **minKey**()
    returns, but does not remove, the smallest key
  - **minElement**()
    returns, but does not remove, the element with smallest key
  - **size**(), **isEmpty**()
- Applications:
  - Standby flyers
  - Auctions
  - Stock market

Total Order Relation

- Keys in a priority queue can be arbitrary objects on which an order is defined
- Two distinct items in a priority queue can have the same key

- Mathematical concept of total order relation $\leq$
  - Reflexive property: $x \leq x$
  - Antisymmetric property: $x \leq y$ & $y \leq x \Rightarrow x = y$
  - Transitive property: $x \leq y$ & $y \leq z \Rightarrow x \leq z$

Comparator ADT

- A comparator compares two objects according to a given total order relation
- A generic priority queue uses an associated comparator
- The comparator is external to the keys being compared
- When the priority queue needs to compare two keys, it uses its comparator
- Methods of the Comparator ADT, all with Boolean return type
  - isLessThan(x, y)
  - isLessThanOrEqualTo(x,y)
  - isEqualTo(x,y)
  - isGreaterThan(x, y)
  - isGreaterThanOrEqualTo(x,y)
  - isComparable(x)

Sorting with a Priority Queue

- We can use a priority queue to sort a set of comparable elements
  1. Insert the elements one by one with a series of **insertItem**(e, e) operations
  2. Remove the elements in sorted order with a series of **removeMin**() operations
- The running time of this sorting method depends on the priority queue implementation

Algorithm ***PQ-Sort***(***S***, ***C***)

Input sequence ***S***, comparator ***C*** for the elements of ***S***

Output sequence ***S*** sorted in increasing order according to ***C***

P ← priority queue with comparator ***C***

while ¬***S.isEmpty*** ()

e ← ***S.remove*** (***S. first*** ())

P.insertItem(***e***, ***e***)

while ¬***P.isEmpty***()

e ← ***P.removeMin***()

S.insertLast(***e***)

Sequence-based Priority Queue

- Implementation with an unsorted sequence
  - Store the items of the priority queue in a sequence, in arbitrary order

- Performance:
  - **insertItem** takes $O(1)$ time since we can insert the item at the beginning or end of the sequence
  - **removeMin**, **minKey** and **minElement** take $O(n)$ time since we have to traverse the entire sequence to find the smallest key

- Implementation with a sorted sequence
  - Store the items of the priority queue in a sequence, sorted by key

- Performance:
  - **insertItem** takes $O(n)$ time since we have to find the place where to insert the item
  - **removeMin**, **minKey** and **minElement** take $O(1)$ time since the smallest key is at the beginning of the sequence

Selection-Sort

- **Selection-sort** is the variation of PQ-sort where the priority queue is implemented with an unsorted sequence

- Running time of Selection-sort:
  1. Inserting the elements into the priority queue with n **insertItem** operations takes $\boldsymbol{O}(n)$ time
  2. Removing the elements in sorted order from the priority queue with n **removeMin** operations takes time proportional to
     $$1 + 2 + \ldots + n$$

- **Selection-sort** runs in $\boldsymbol{O}(\boldsymbol{n}^2)$ time

Insertion-Sort

- **Insertion-sort** is the variation of PQ-sort where the priority queue is implemented with a sorted sequence

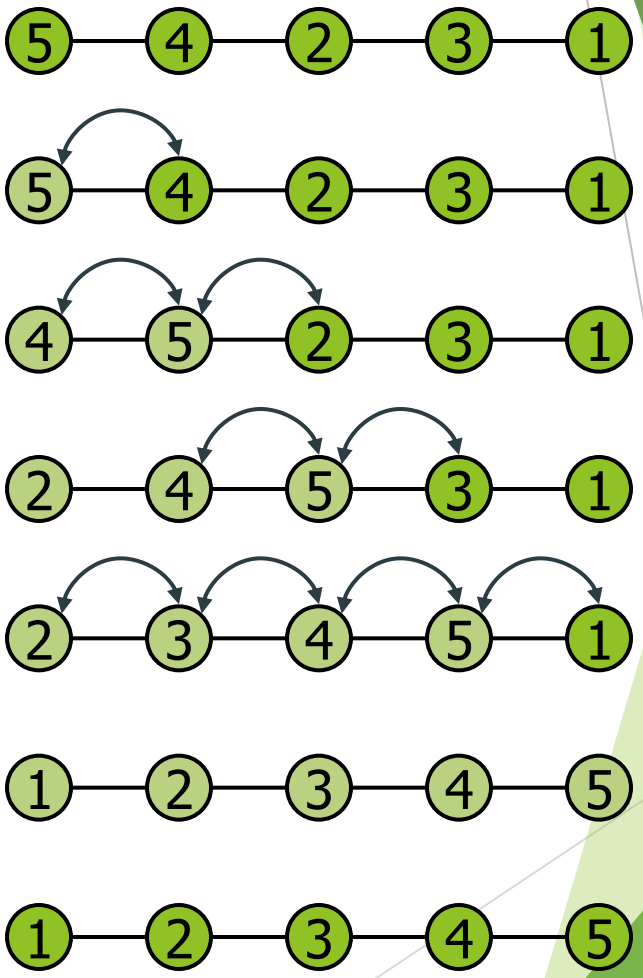
- Running time of Insertion-sort:
  1. Inserting the elements into the priority queue with n **insertItem** operations takes time proportional to
     $$1 + 2 + \ldots + n$$
  2. Removing the elements in sorted order from the priority queue with a series of n **removeMin** operations takes $\boldsymbol{O}(n)$ time

- **Insertion-sort** runs in $\boldsymbol{O}(n^2)$ time

In-place Insertion-sort

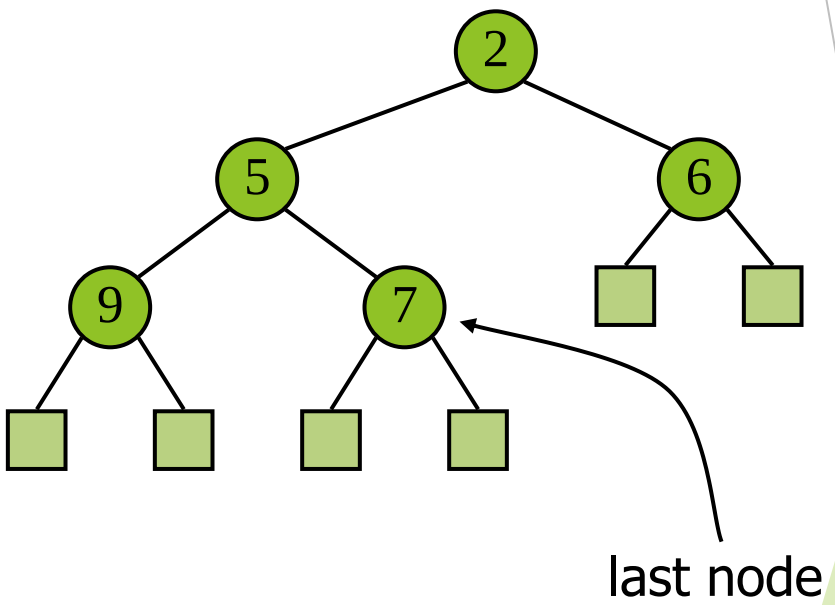
- Instead of using an additional data structure, we can implement selection-sort and insertion-sort in-place
- A portion of the input sequence itself serves as the priority queue
- For in-place insertion-sort
  - We keep the initial portion of the sequence sorted
  - We can use **swapElements** instead of modifying the sequence

Save space !!



What is a heap

- A heap is a binary tree storing keys at its internal nodes and satisfying the following properties:
  - **Heap-Order**: for every internal node v other than the root, $key(v) \geq key(parent(v))$
  - **Complete Binary Tree**: let h be the height of the heap
    - for $i = 0, \ldots, h-1$, there are 2^i nodes of depth i
    - at depth $h-1$, the internal nodes are to the left of the external nodes
    - The last node of a heap is the rightmost internal node of depth $h-1$

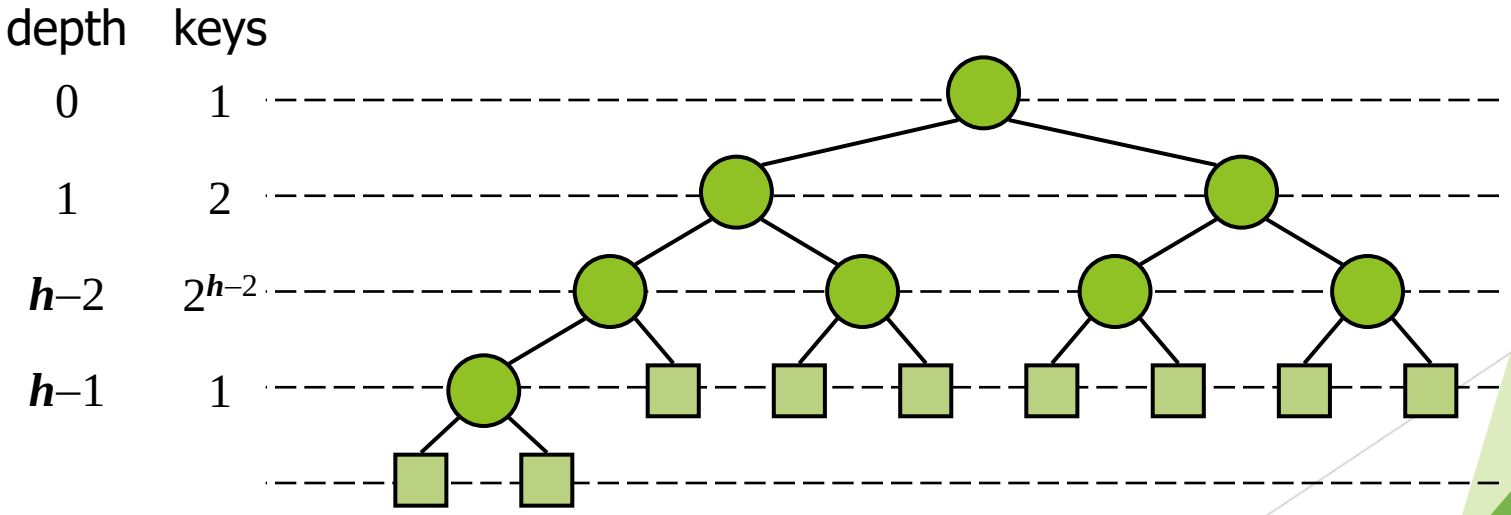


Height of a Heap

- Theorem: A heap storing n keys has height $O(\log n)$

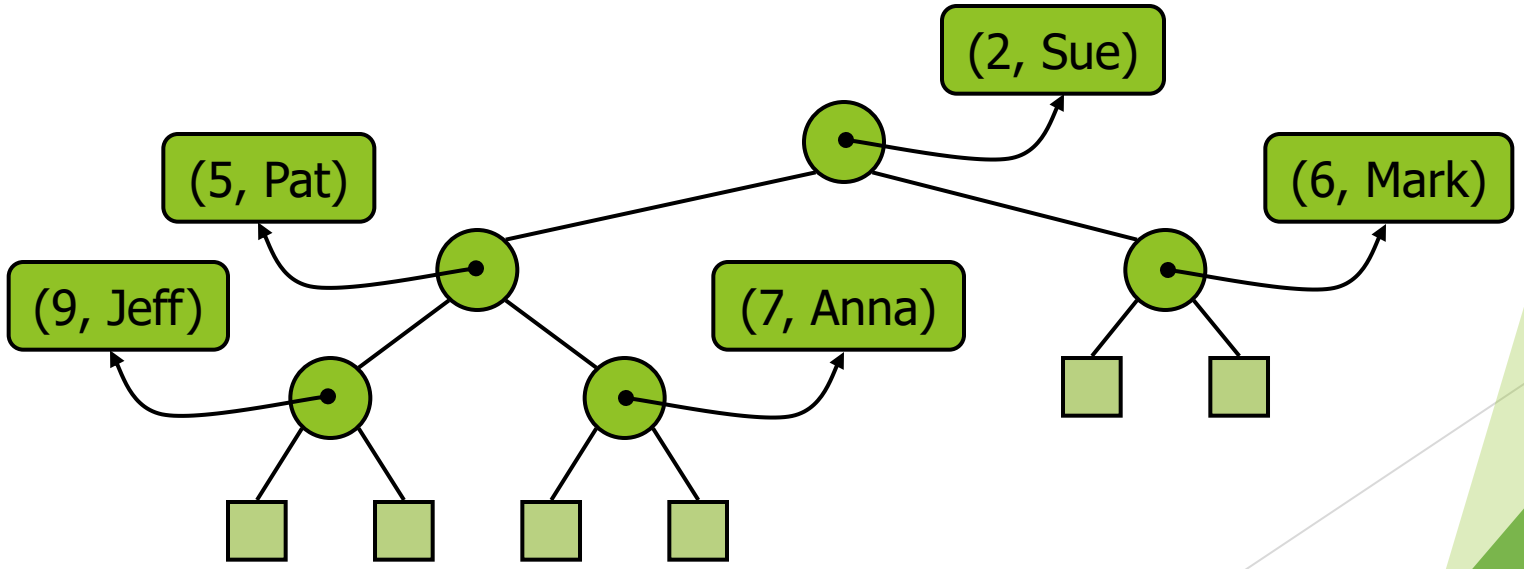
  Proof: (we apply the complete binary tree property)

  - Let h be the height of a heap storing n keys
  - Since there are 2^i keys at depth $i = 0, \ldots, h-2$ and at least one key at depth $h-1$, we have $n \geq 1 + 2 + 4 + \ldots + 2^{h-2} + 1$
  - Thus, $n \geq 2^{h-1}$, i.e., $h \leq \log n + 1$



Heaps and Priority Queues

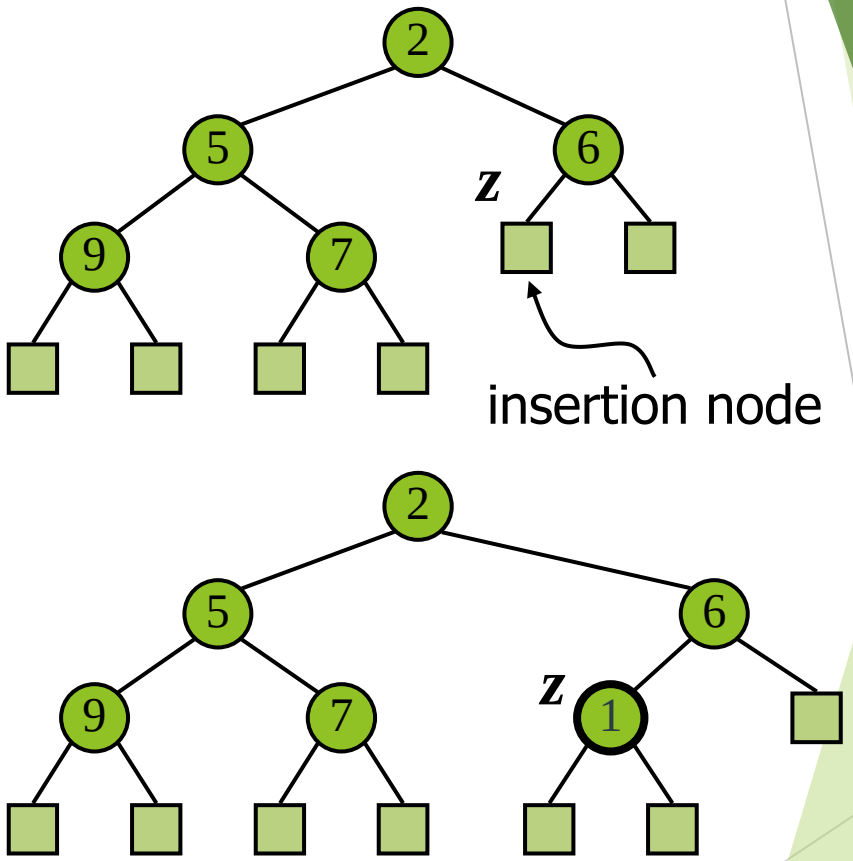
- We can use a heap to implement a priority queue
- We store a (key, element) item at each internal node
- We keep track of the position of the last node
- For simplicity, we show only the keys in the following slides



Insertion into a Heap

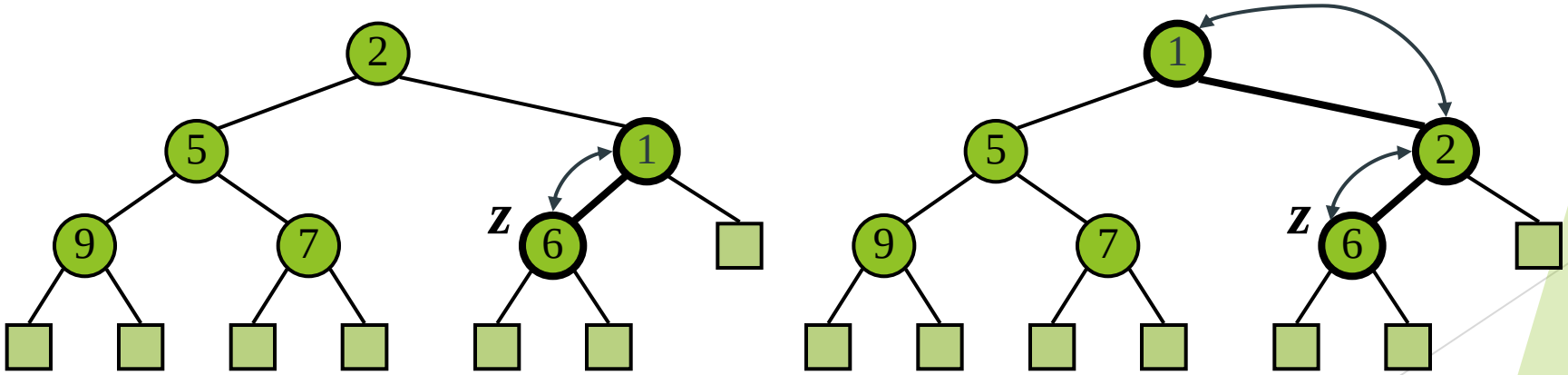
- Method **insertItem** of the priority queue ADT corresponds to the insertion of a key k to the heap

- The insertion algorithm consists of three steps
  - Find the insertion node z (the first external node)
  - Store k at z and expand z into an internal node
  - Restore the heap-order property (discussed next)



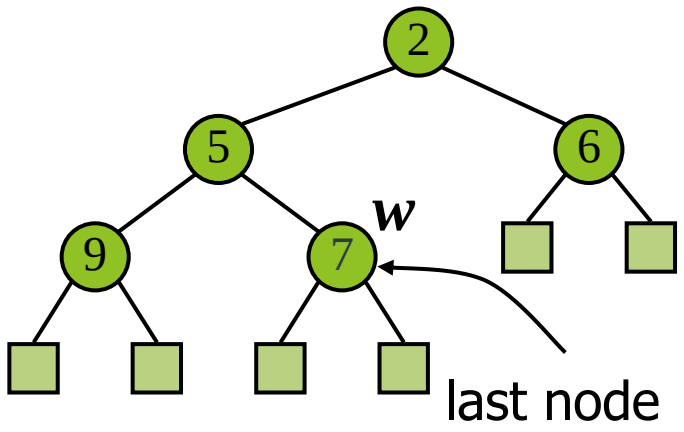
Upheap

- After the insertion of a new key k, the heap-order property may be violated
- Algorithm **upheap** restores the heap-order property by swapping k along an upward path from the insertion node
- **Upheap** terminates when the key k reaches the root or a node whose parent has a key smaller than or equal to k
- Since a heap has height $O(\log n)$, upheap runs in $O(\log n)$ time

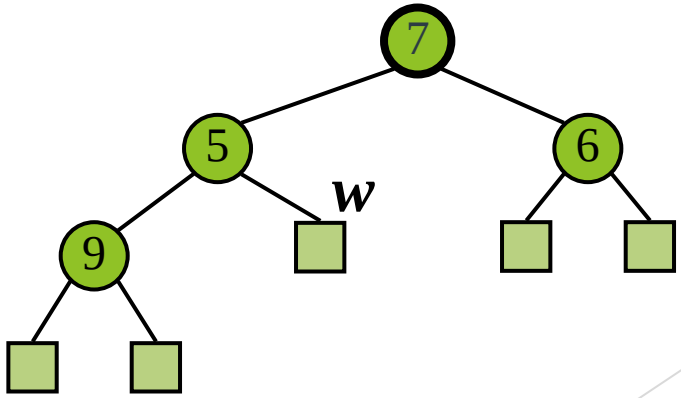


Removal from a Heap

- Method **removeMin** of the priority queue ADT corresponds to the removal of the *root* key from the heap

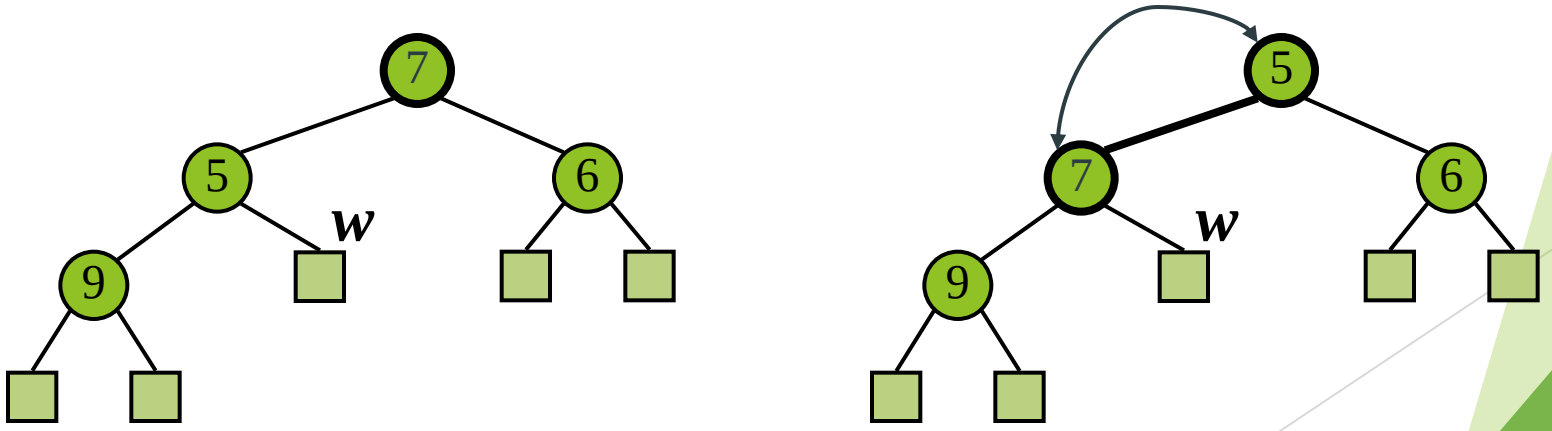


- The removal algorithm consists of three steps
  - Replace the *root* key with the key of the last node w
  - Compress w and its children into a leaf
  - Restore the heap-order property (discussed next)



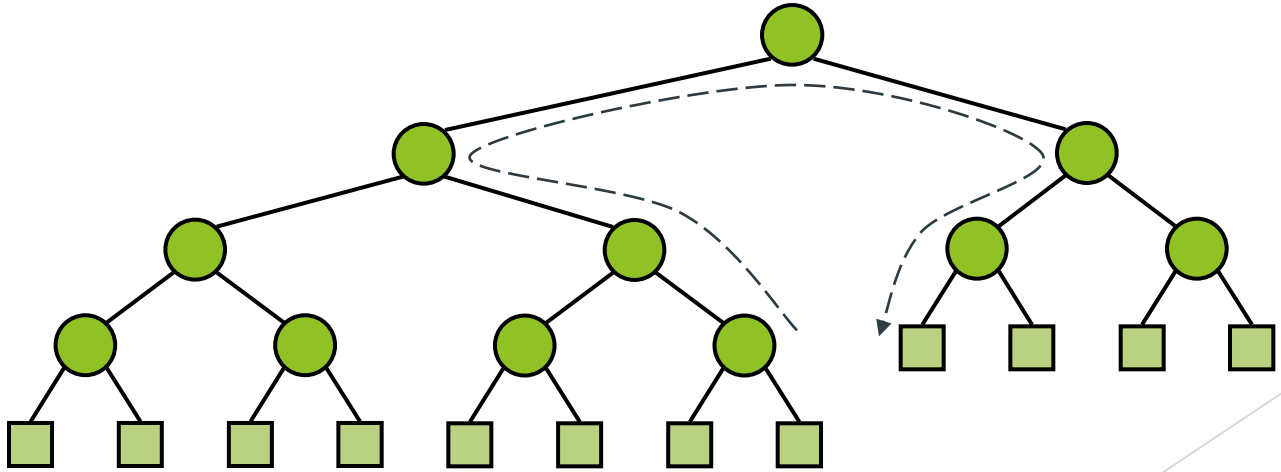
Downheap

- After replacing the root key with the key k of the last node, the heap-order property may be violated
- Algorithm **downheap** restores the heap-order property by swapping key k along a downward path from the root (between the two children of a node, consider the one with smaller key)
- **Downheap** terminates when key k reaches a leaf or a node whose children have keys greater than or equal to k
- Since a heap has height $O(\log n)$, downheap runs in $O(\log n)$ time



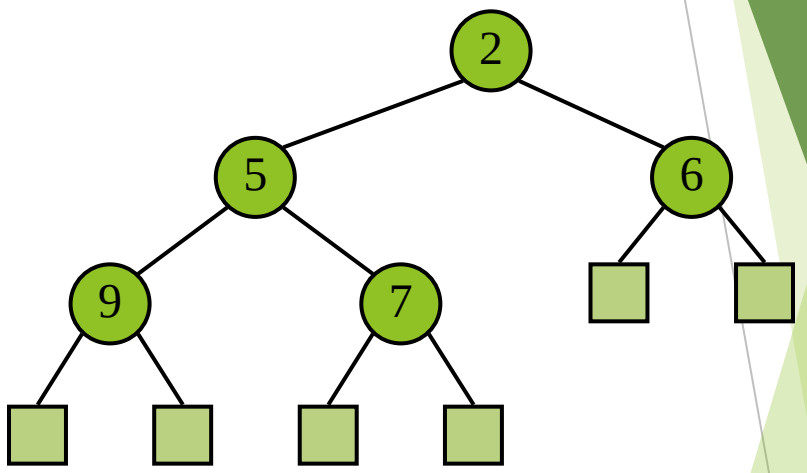
Finding the first external Node: List-based

- The insertion node can be found by traversing a path of $O(\log n)$ nodes
  - Assuming we have a pointer to the last node.
  - Starting from it, do 3 steps:
    - (1) While the current node is a right child, go to the parent node
    - (2) If the current node is a left child, go to the right child
    - (3) While the current node is internal, go to the left child
- Similar algorithm for updating the pointer to the last node after a removal



Finding the Last Node: Vector-based

- We can represent a heap with n keys by means of a vector of length $n + 1$
- For the node at rank i
  - the left child is at rank $2i$
  - the right child is at rank $2i + 1$
- Links between nodes are not explicitly stored
- The leaves are not represented
- The cell at rank 0 is not used
- Operation **insertItem** corresponds to inserting at rank $n + 1$
- Operation **removeMin** corresponds to removing at rank 1



	2	5	6	9	7
0	1	2	3	4	5

Heap-Sort

- Consider a priority queue with n items implemented by means of a heap
  - the space used is $O(n)$
  - methods **insertItem** and **removeMin** take $O(\log n)$ time
  - methods **size**, **isEmpty**, **minKey**, and **minElement** take time $O(1)$ time
- Using a heap-based priority queue, we can sort a sequence of n elements in $O(n \log n)$ time
- The resulting algorithm is called heap-sort
- Heap-sort is much faster than quadratic sorting algorithms, such as insertion-sort and selection-sort

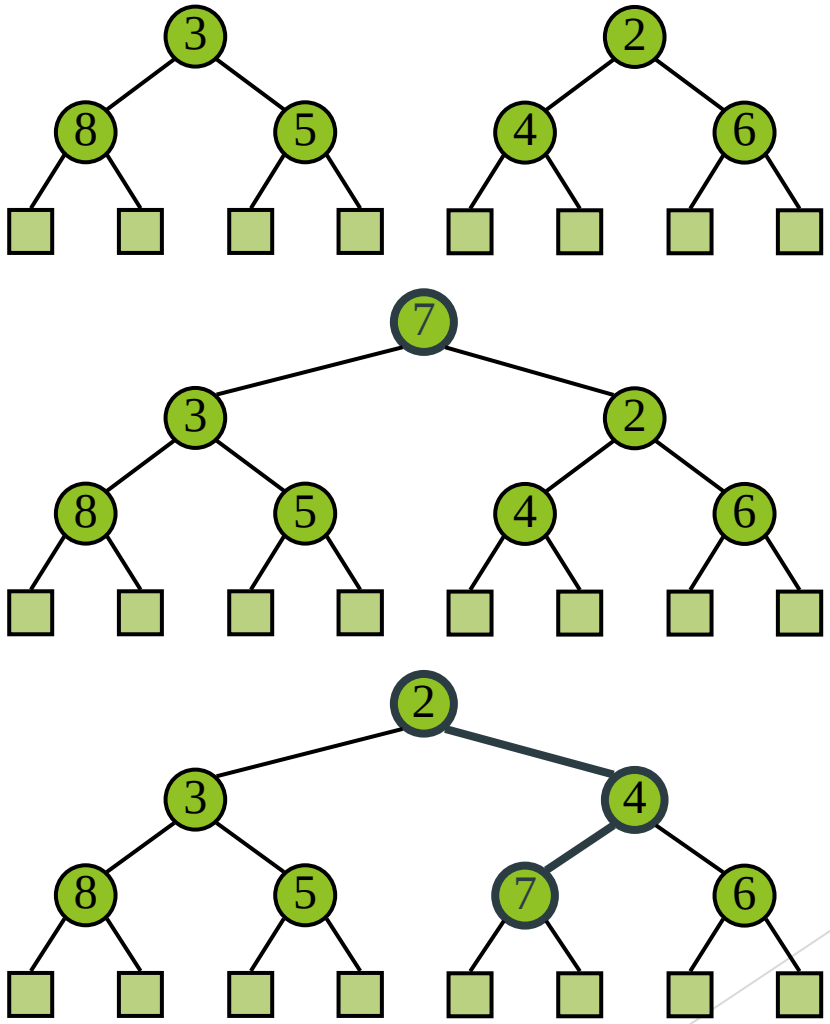
```
Algorithm PQ-Sort(S, C)
    Input sequence S, comparator C for the
    elements of S
    Output sequence S sorted  in increasing
    order according to C
    P ← priority queue with
        comparator C
    while ¬S.isEmpty ()
        e ← S.remove (S. first ())
        P.insertItem(e, e)
    while ¬P.isEmpty()
        e ← P.removeMin()
        S.insertLast(e)
```

The old PQ-Sort runs in $O(n^2)$ time

The new heapSort runs in $O(n \log n)$ time

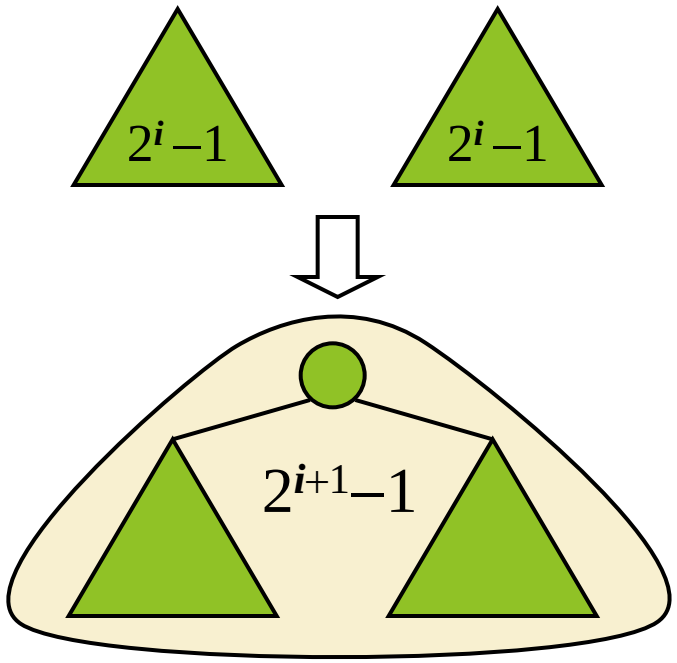
Merging Two Heaps

- We are given two heaps and a key k
- We create a new heap with the root node storing k and with the two heaps as subtrees (children)
- We perform **downheap** to restore the heap-order property



Bottom-up Heap Construction

- We can construct a heap storing n given keys using a bottom-up construction with $\log n$ phases
- In phase i, pairs of heaps with $2^i - 1$ keys are merged into heaps with $2^{i+1} - 1$ keys



Algorithm ***BottomUpHeap***(***S***)

Input sequence ***S*** of $n=2^h-1$ keys

Output A heap **T** storing the keys

If **S** is empty **then**

return an empty heap (external node)

Remove the first key, **k**, from **S**

Split **S** equally into two sequences **S1** and **S2**

T1 ← BottomUpHeap(S1)

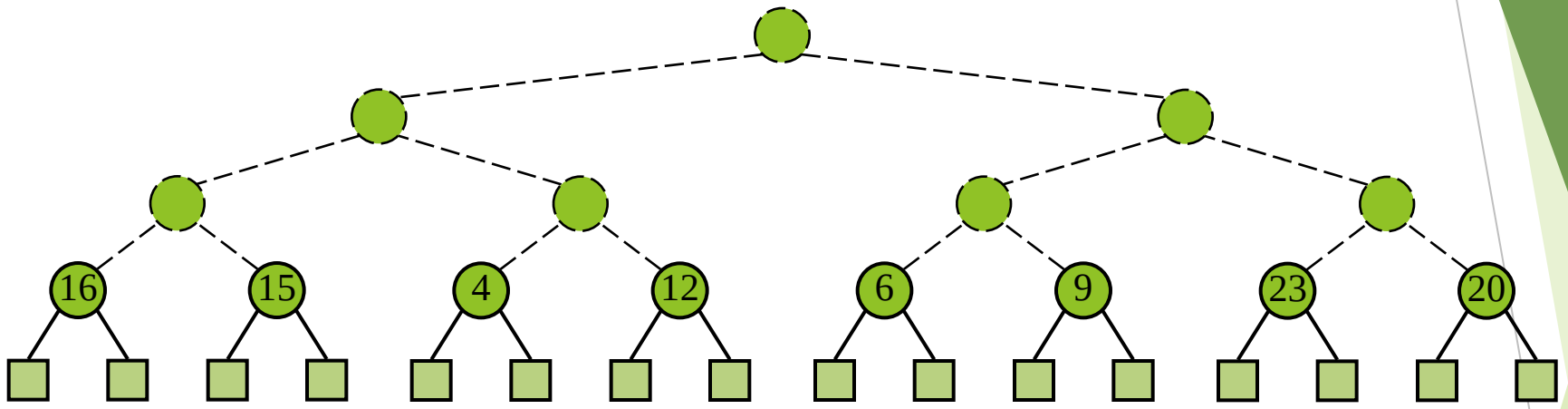
T2 ← BottomUpHeap(S2)

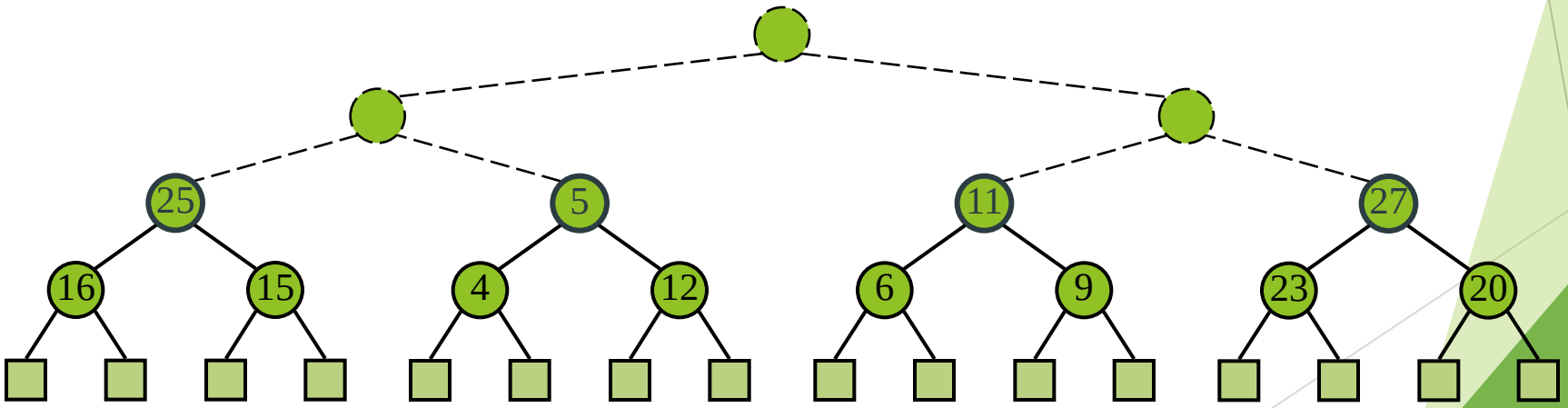
Create binary tree **T** with root r storing **k**, left subtree **T1** and right subtree **T2**

Perform a down-heap bubbling from the root **r** of **T**, if necessary

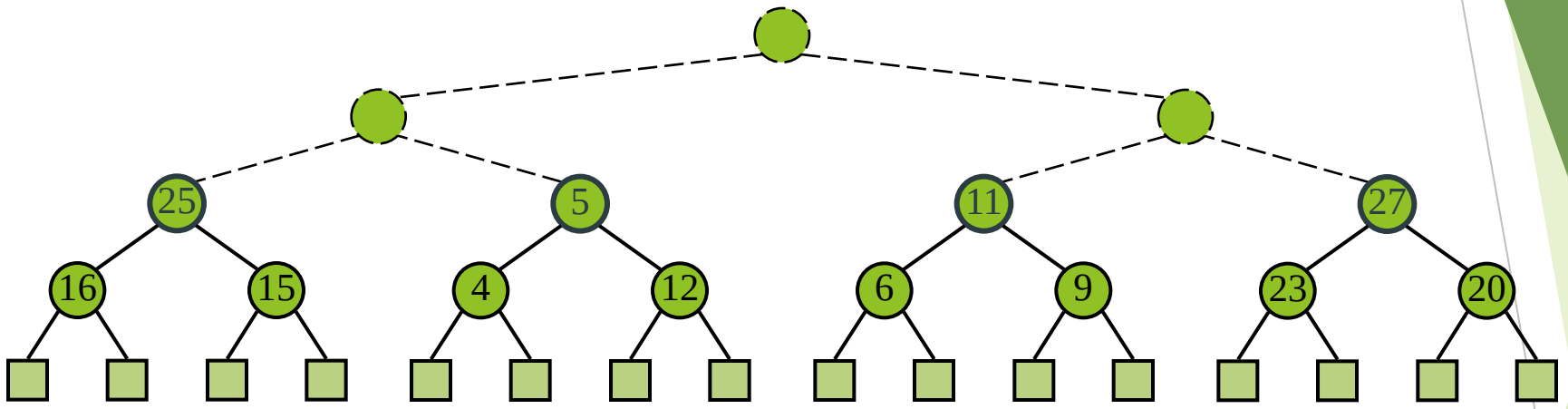
Return T

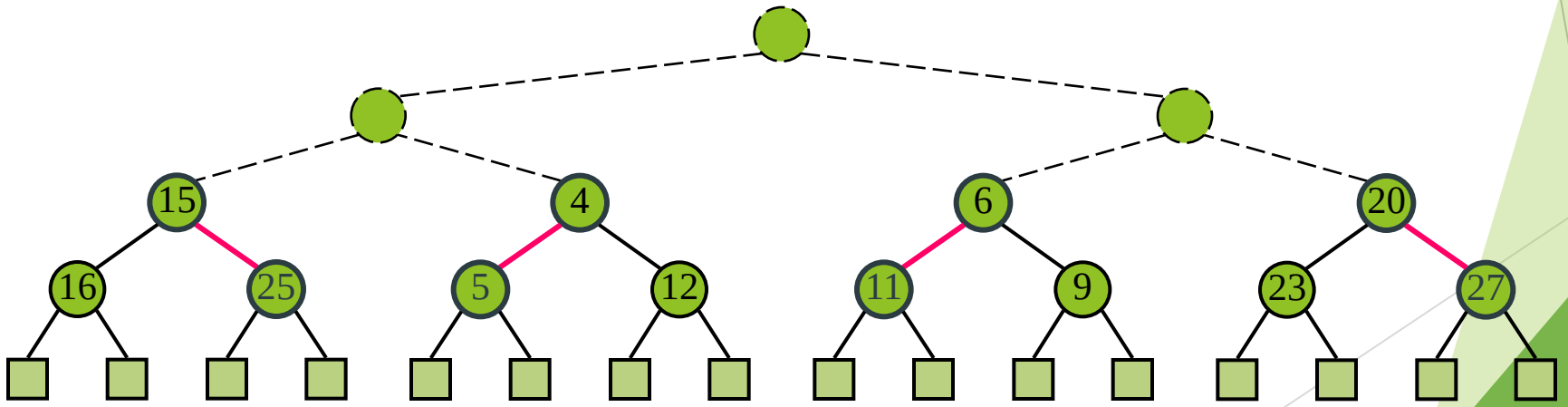
Example



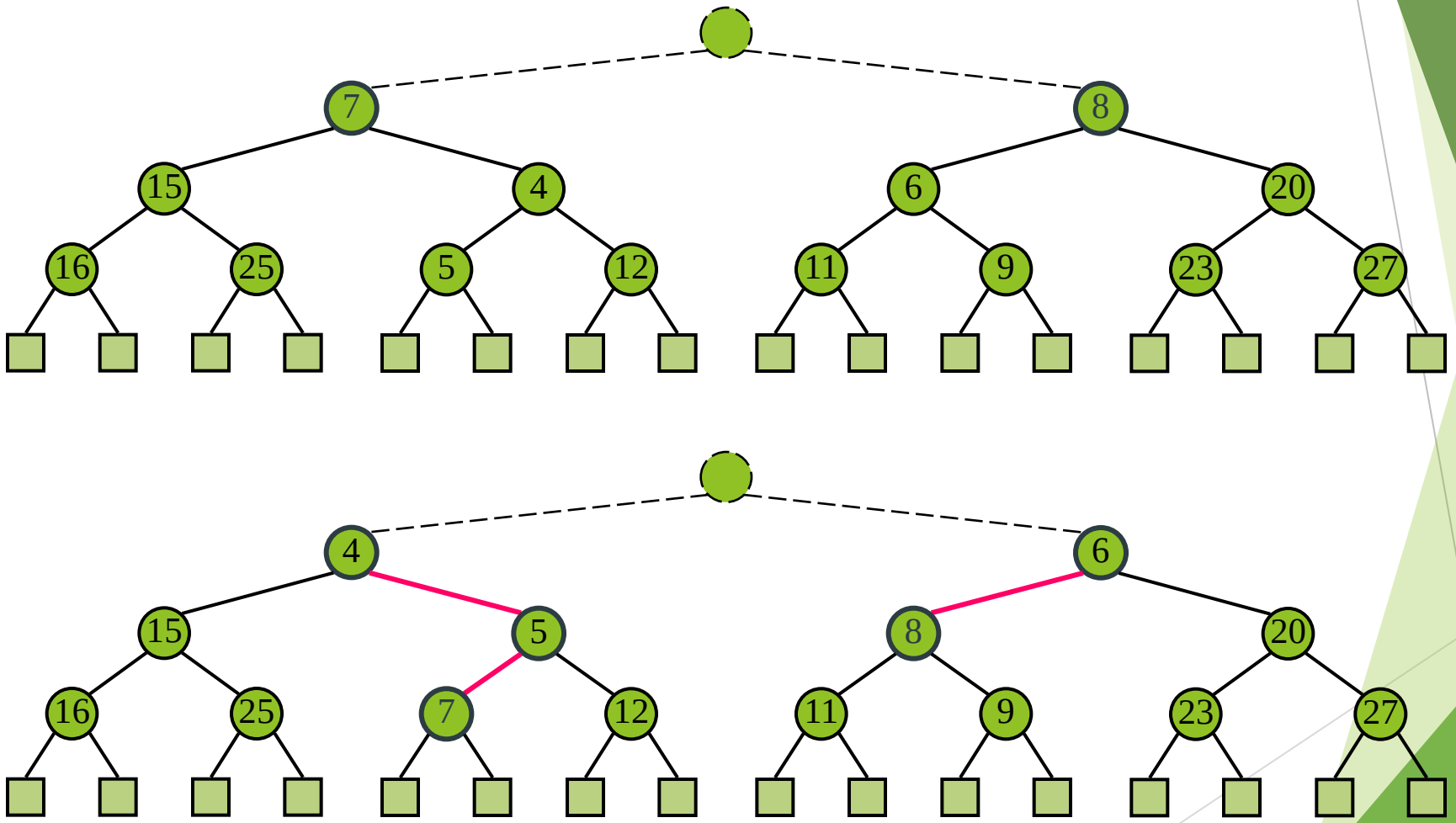


Example (contd.)

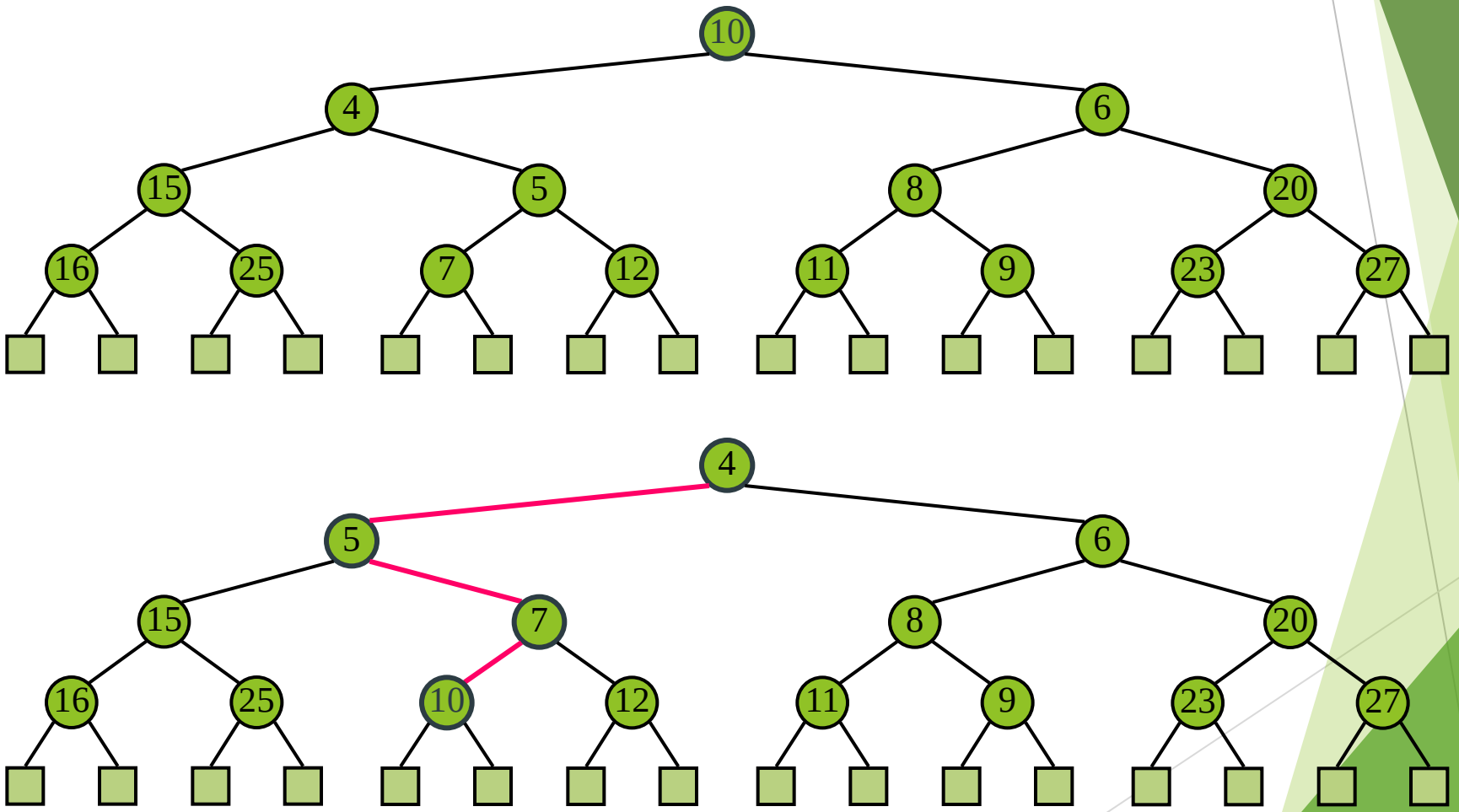




Example (contd.)



Example (end)



Some Math

- Leaf nodes no operation needed as each single node is a heap in itself.
  - Number of such nodes are CEIL(n/2)
  - Height of these nodes = 0
- Internal nodes at lowest level.
  - Number of such nodes are CEIL(n/4)
  - Heigh of subtrees are =1
- Internal nodes at lowest+1 level.
  - Number of such nodes are CEIL(n/8)
  - Heigh of subtrees are =2
- ….
- Internal nodes at heighest level.
  - Number of such nodes are CEIL(n/2^k = 1) is 1(root)
  - Heigh of subtree is $\log_2 n$

Arithmetic Geometric Series

Cost of Build Heap

$$S = 1*n/2 + 2*n/4 + 3*n/8 + 4*n/16 + \text{-------} + h\, n/2^h$$

$$S = n/2 + 2n/4 + 3n/8 + 4n/16 + \text{-------} + h\, n/2^h$$

$d = 1$

$r = ½$

$$S/2 = n/4 + 2n/8 + 3n/16 + \text{-------} + (h-1)n/2^h + h\, n/2^{h+1}$$

$$S - S/2 = n/2 + n/4 + n/8 + n/16 + \text{-----} + n/2^h - h\, n/2^{h+1}$$

$$S = n + n/2 + n/4 + n/8 + \text{-----} + n/2^{h-1} - h\, n/2^h$$

h is the height of a tree replace it with $\log_2 n$

$$S = n(1 + ½ + (½)^2 + + (½)^3 + (½)^{\log_2 n-1}) - \log_2 n$$

$$S = 2n\,(1 - (½)^{\log_2 n - 1}) - \log_2 n$$

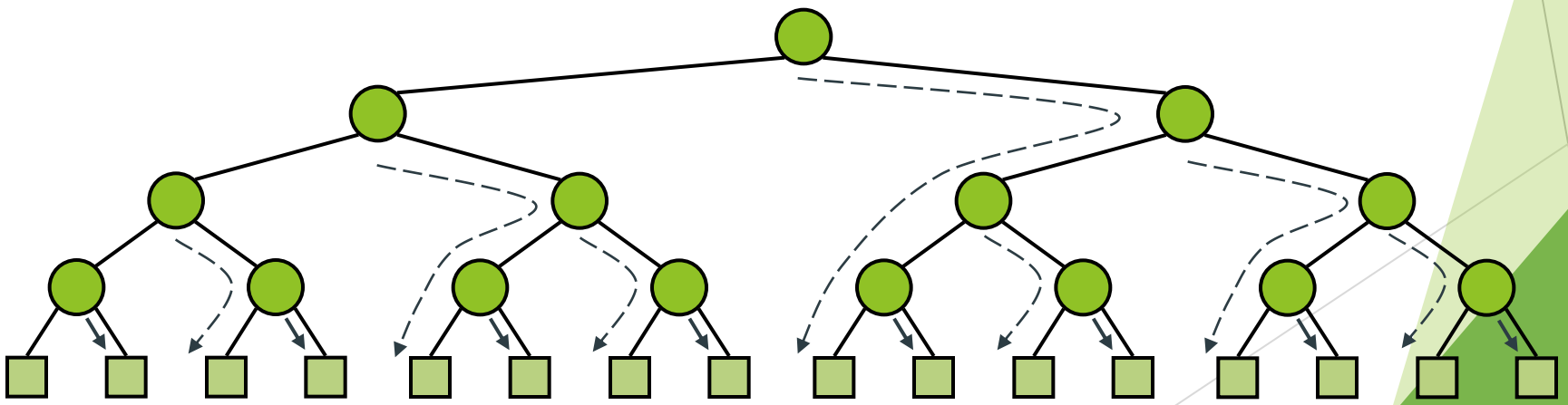
$$S = 2n(1 - 2/n) - \log_2 n = 2n - \log_{2n} - 4 = O(n)$$

In Other Words, using Identity:

$$N = \sum_{i=0}^{log(n)} \frac{n}{2^{i+1}} i = \frac{n}{2} \left(\sum_{i=0}^{log(n)} i \left(1/2\right)^i \right) \leq \frac{n}{2} \left(\sum_{i=0}^{\infty} i \left(1/2\right)^i \right)$$

$$\sum_{i=1}^{\infty} i x^i = \frac{x}{\left(1-x\right)^2}$$

$$N \leq \frac{n}{2} \left(2\right) = n$$

Analysis

- We visualize the worst-case time of a downheap with a proxy path that goes first right and then repeatedly goes left until the bottom of the heap (this path may differ from the actual **downheap** path)
- Since each node is traversed by at most two proxy paths, and any two paths do NOT share edges, the total number of edges of all the paths is no more than $2n$
- Thus, bottom-up heap construction runs in $O(n)$ time
- Bottom-up heap construction is faster than n successive insertions/removals using **heap-sort**